

Procedure for Using Shared Memory

- Find a *key*: Unix uses this key for identifying shared memory segments
- Use `shmget()` to allocate a shared memory
- Use `shmat()` to attach a shared memory to an address space
- Use `shmdt()` to detach a shared memory from an address space
- Use `shmctl()` to deallocate a shared memory

Project 1

➤ Header (incomplete)

```
#include <sys/ipc.h>
```

```
#include <sys/shm.h>
```

```
#include <unistd.h>
```

```
#include <errno.h>
```

```
/* key number */
```

```
#define SHMKEY ((key_t) 1497)
```

Key

- Unix uses this key for identifying shared memory segments.
 - A key is a value of type `key_t`
 - There are three ways to generate a key
 - Do it yourself
 - Use function `ftok()`
 - Ask the system to provide a private key

Key

- Do it yourself:
 - `#define SHMKEY ((key_t) 1497)`
- Use `ftok()` to generate one for you
 - `key_t = ftok(char *path, int ID);`
 - `path` is a path name (e.g., `“./”`)
 - `ID` is an integer (e.g., `‘a’`)
 - Function `ftok()` returns a key of type `key_t`: `Key = ftok(“./”, ‘x’)`
- Keys are global entities. If other processes know your key, they can access your shared memory.
- Ask the system to provide a private key using `IPC_PRIVATE`

Project 1

- shared memory

```
typedef struct
```

```
{
```

```
    int value;
```

```
} shared_mem;
```

```
shared_mem *total;
```

- process1 increases the value of shared variable "total" * by some number
- process2
- process3
- process4

Project 1

➤ Main function

```
int  shmid, pid1, pid2, pid3, pid4, ID, status;
```

```
char *shmadd;
```

```
shmadd = (char *) 0;
```

➤ `/* Create and connect to a shared memory segment*/`

```
if ((shmid = shmget (SHMKEY, sizeof(int), IPC_CREAT | 0666)) < 0){
```

```
    perror ("shmget");
```

```
    exit (1);} 
```

```
if ((total = (shared_mem *) shmat (shmid, shmadd, 0)) ==  
(shared_mem *) -1) {
```

```
    perror ("shmat");
```

```
    exit (0);} 
```

Project 1

- Initialize shared memory to 0

`total->value = 0;`

- Create processes

`if ((pid1 = fork()) == 0)`

`process1();`

- Create additional processes

Project 1

- Parent wait for child processes to finish and print ID of each child. Three ways:
 - `wait(&status)`
 - `wait(NULL)`
 - `waitpid(pid1, NULL, 0);`
 - `printf("Child with pid %d has just exited.\n", pid1);`

Project 1

- To detach a shared memory, use `shmdt(total);`
 - `total` is the pointer returned by `shmat()`

```
if (shmdt(total) == -1) {  
    perror ("shmdt");  
    exit (-1);  
}
```
- To remove a shared memory, use `shmctl(shmid, IPC_RMID, NULL);`
 - `shmid` is the shared memory ID returned by `shmget()`.
- After a shared memory is removed, it no longer exists.

Notes

- If you did not remove your shared memory segments (e.g., program crashes before the execution of `shmctl()`), they will be in the system forever. This will degrade the system performance.
- Use the `ipcs` command to check if you have shared memory segments left in the system.
- Use the `ipcrm` command to remove your shared memory segments.